Woche 05 – Übungen

Orakel, Phase Kickback, Deutsch–Jozsa und Bernstein–Vazirani

Softwarestand für Aufgaben 7–16: Python, NumPy, optional `pytest` sowie `qiskit[all]~=2.5.1` oder neuer. Verwendet werden `QuantumCircuit`, `qiskit.quantum_info.Statevector` und `Operator`. Dokumentation: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector, https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Operator und https://quantum.cloud.ibm.com/docs/en/guides/bit-ordering.

Registerkonvention: Mathematisch ist $x = (x_{n-1}, \ldots, x_0)$ und $|x\rangle = |x_{n-1} \cdots x_0\rangle$. In Qiskit repräsentiert q_j das Bit x_j; das Zielqubit ist q_n. Ein vollständiges Basislabel besitzt die Reihenfolge $|y\, x_{n-1} \cdots x_0\rangle$. Der Integerindex ist $x + 2^n y$.

Lernziele

LZ1 Boolesche Funktionen als reversible Bit- und Phasenorakel darstellen und prüfen.
LZ2 Phase Kickback, $H^{\otimes n}$ und Charakterorthogonalität herleiten.
LZ3 Deutsch–Jozsa und Bernstein–Vazirani algebraisch analysieren.
LZ4 Orakel und Algorithmen in Python/Qiskit reproduzierbar implementieren.
LZ5 Promise-, Phasen-, Test- und Bitordnungsfehler diagnostizieren.
LZ6 Query-Komplexität, Orakelkosten und physikalisches Phase Kickback vergleichen.

Grundlagen und mathematische Sicherheit

Aufgabe 1 – Ein irreversibles f als reversibles Orakel

Schwierigkeit: Basis **Arbeitsform:** Papier
Zeit: 25 Minuten **Lernziel(e):** LZ1

Für $x = (x_1, x_0) \in \{0,1\}^2$ sei $f(x) = x_1 x_0$ und

$$U_f\,|x_1 x_0\rangle\,|y\rangle = |x_1 x_0\rangle\,|y \oplus x_1 x_0\rangle\,.$$

a) Erstellen Sie die Wahrheitstabelle von f.

b) Geben Sie die Wirkung auf alle acht Basiszustände $|x_1 x_0 y\rangle$ an.

c) Schreiben Sie die 8×8-Permutationsmatrix in der Basis $(|000\rangle, \ldots, |111\rangle)$.

d) Zeigen Sie $U_f^2 = I_8$.

e) Begründen Sie, warum $x \mapsto f(x)$ nicht unitär implementierbar ist, die erweiterte Abbildung aber schon.

f) Welches kontrollierte Gatter realisiert dieses Orakel?

Hinweis / Selbstkontrolle

Die Matrix ist die Einheitsmatrix mit vertauschten Zuständen $|110\rangle$ und $|111\rangle$. Das Gatter ist ein doppelt kontrolliertes X.

Aufgabe 2 – Projektorform, Eigenwerte und Determinante

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 30 Minuten **Lernziel(e):** LZ1, LZ2

Sei

$$U_f = \sum_{x\in\{0,1\}^n} |x\rangle\langle x| \otimes X^{f(x)}, \qquad m = \#\{x : f(x) = 1\}, \quad N = 2^n.$$

a) Leiten Sie $U_f^\dagger = U_f$ und $U_f^2 = I$ her.

b) Zeigen Sie, dass $|x\rangle\,|+\rangle$ stets Eigenwert $+1$ besitzt.

c) Bestimmen Sie den Eigenwert von $|x\rangle\,|-\rangle$.

d) Bestimmen Sie die Vielfachheiten von $+1$ und -1.

e) Berechnen Sie $\mathrm{Tr}(U_f)$ und $\det(U_f)$.

f) Prüfen Sie die Formeln für konstante und balancierte Funktionen.

Hinweis / Selbstkontrolle

Die -1-Vielfachheit ist m. Daher $\det(U_f) = (-1)^m$ und $\mathrm{Tr}(U_f) = 2(N - m)$.

Aufgabe 3 – Phase Kickback mit allgemeinem Zielzustand

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 35 Minuten **Lernziel(e):** LZ1, LZ2

Sei

$$|\tau\rangle = a\,|+\rangle + b\,|-\rangle\,, \qquad |a|^2 + |b|^2 = 1, \qquad |\psi\rangle = \sum_x \alpha_x\,|x\rangle\,.$$

a) Leiten Sie $U_f(|\psi\rangle\,|\tau\rangle)$ vollständig her.

b) Unter welchen Bedingungen an a, b entsteht für jedes f wieder ein Produktzustand?

c) Welche Bedingung liefert ein nichttriviales Phasenorakel?

d) Setzen Sie $|\psi\rangle = N^{-1/2}\sum_x |x\rangle$ und ein balanciertes f. Berechnen Sie die reduzierte Dichtematrix des Zielregisters in der Basis $(|+\rangle\,, |-\rangle)$.

e) Berechnen Sie deren Reinheit. Für welche a, b ist die Verschränkung maximal?

f) Erklären Sie die Spezialrolle von $|-\rangle$.

Hinweis / Selbstkontrolle

Für balanciertes f verschwindet $N^{-1}\sum_x(-1)^{f(x)}$. Die reduzierte Zielmatrix ist diagonal mit $|a|^2$ und $|b|^2$.

Aufgabe 4 – n-Qubit-Hadamard und Charakterorthogonalität

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 35 Minuten **Lernziel(e):** LZ2

a) Leiten Sie

$$H^{\otimes n}\,|x\rangle = \frac{1}{\sqrt{2^n}}\sum_z (-1)^{x\cdot z}\,|z\rangle$$

her.

b) Entwickeln Sie $H^{\otimes 3}\,|011\rangle$ vollständig.

c) Beweisen Sie $\sum_x (-1)^{x\cdot w} = 2^n\delta_{w,0^n}$.

d) Zeigen Sie damit $(H^{\otimes n})^2 = I$.

e) Für $\chi_s(x) = (-1)^{s\cdot x}$ zeigen Sie

$$\frac{1}{N}\sum_x \chi_s(x)\chi_t(x) = \delta_{s,t}.$$

f) Interpretieren Sie die χ_s als orthonormale Funktionsbasis.

Hinweis / Selbstkontrolle

Für Teil b) hängt das Vorzeichen von $z_1 \oplus z_0$ ab. In Teil d) tritt $w = x \oplus z$ auf.

Aufgabe 5 – Deutsch–Jozsa jenseits der Standardbeispiele

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 40 Minuten **Lernziel(e):** LZ2, LZ3, LZ5

Für $n = 3$ seien

$$f_0(x) = 0, \quad f_p(x) = x_2 \oplus x_0, \quad f_{\text{maj}}(x) = \mathbf{1}[x_2 + x_1 + x_0 \geq 2], \quad f_{\text{and}}(x) = x_2x_1x_0.$$

a) Klassifizieren Sie jede Funktion als konstant, balanciert oder außerhalb der Promise.

b) Berechnen Sie jeweils $a_{000} = 8^{-1}\sum_x (-1)^{f(x)}$.

c) Bestimmen Sie für f_p den vollständigen Endzustand.

d) Berechnen Sie das vollständige Walsh-Spektrum von f_{maj}.

e) Geben Sie dessen Messverteilung an.

f) Berechnen Sie $p(000)$ für f_{and}.

Hinweis / Selbstkontrolle

Majorität liefert je $1/4$ auf $001, 010, 100, 111$. Für f_{and} gilt $p(000) = 9/16$.

Aufgabe 6 – Bernstein–Vazirani und affiner Offset

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 35 Minuten **Lernziel(e):** LZ2, LZ3

Sei $f_{s,b}(x) = b \oplus s \cdot x$.

a) Leiten Sie $O_{s,b} = (-1)^b \bigotimes_j Z^{s_j}$ her.

b) Zeigen Sie mit Charakterorthogonalität, dass der Algorithmus $(-1)^b\,|s\rangle$ erzeugt.

c) Leiten Sie dasselbe Ergebnis aus $HZH = X$ her.

d) Rechnen Sie den Algorithmus für $s = 11010$, $b = 1$ aus.

e) Geben Sie ein klassisches exaktes Abfrageschema für s an.

f) Wie lässt sich b klassisch zusätzlich bestimmen, und warum liefert das Phasenexperiment b nicht?

Hinweis / Selbstkontrolle

$f_{s,b}(0^n) = b$. Im Quantenalgorithmus ist $(-1)^b$ global.

Python und Qiskit: Orakel implementieren und testen

Aufgabe 7 – Walsh–Hadamard-Spektren mit NumPy

Schwierigkeit: Basis **Arbeitsform:** Python
Zeit: 50 Minuten **Lernziel(e):** LZ2–LZ4

Implementieren Sie zunächst direkt

$$\widehat{g}(z) = \frac{1}{N} \sum_{x} g(x)(-1)^{x \cdot z}$$

für einen Vektor `g` der Länge $N = 2^n$.

a) Implementieren Sie `bit_dot(x, z)` für Integer-Bitstrings.

b) Implementieren Sie `walsh_direct(g)` mit Laufzeit $O(N^2)$.

c) Implementieren Sie die Fast-Walsh–Hadamard-Transformation:

```python
def walsh_fast(g):
    a = np.asarray(g, dtype=complex).copy()
    h = 1
    while h < len(a):
        for start in range(0, len(a), 2 * h):
            u = a[start:start + h].copy()
            v = a[start + h:start + 2 * h].copy()
            a[start:start + h] = u + v
            a[start + h:start + 2 * h] = u - v
        h *= 2
    return a / len(a)
```

d) Vergleichen Sie beide Varianten für zufällige Vorzeichenvektoren bis $n = 8$.

e) Prüfen Sie Parseval: $\sum_z |\widehat{g}(z)|^2 = 1$.

f) Reproduzieren Sie die Spektren einer konstanten Funktion, der Parität $s = 101$ und der Drei-Bit-Majorität.

g) Messen Sie Laufzeiten. Warum ist die schnelle klassische Transformation kein Ersatz für die quantummechanische Query-Aussage?

Hinweis / Selbstkontrolle

Die schnelle Variante benötigt $O(N \log N) = O(n2^n)$ Operationen und erwartet die vollständige Wahrheitstabelle. Der Quantenalgorithmus erhält kohärenten Orakelzugriff.

Aufgabe 8 – Qiskit: affine Bit-Orakel mit Basisvertrag

Schwierigkeit: Basis **Arbeitsform:** Qiskit
Zeit: 50 Minuten **Lernziel(e):** LZ1, LZ4, LZ5

Vervollständigen und testen Sie:

```python
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector, Operator


def affine_value(x: int, secret: str, b: int) -> int:
    s = int(secret, 2)
    parity = bin(s & x).count("1") % 2
    return b ^ parity
```

```python

def affine_oracle(secret: str, b: int = 0) -> QuantumCircuit:
    n = len(secret)
    target = n
    oracle = QuantumCircuit(n + 1, name="U_f")
    if b:
        oracle.x(target)
    for j, bit in enumerate(reversed(secret)):
        if bit == "1":
            oracle.cx(j, target)
    return oracle
```

a) Begründen Sie `reversed(secret)`.

b) Testen Sie für `secret="1011"` und beide b alle Basiszustände. Verwenden Sie

$$i_{\text{in}} = x + 2^n y, \qquad i_{\text{out}} = x + 2^n (y \oplus f(x)).$$

c) Erzeugen Sie die Basiszustände mit

```python
Statevector.from_int(index, dims=(2,) * (n + 1))
```

und vergleichen Sie nach `evolve(oracle)` die vollständigen Amplitudenvektoren.

d) Prüfen Sie numerisch $U^\dagger U = I$ und $U^2 = I$ über `Operator(oracle).data`.

e) Mutieren Sie ein aktives Secret-Bit. Welcher Test schlägt fehl?

f) Dokumentieren Sie Stringposition, mathematisches Bit s_j und Qiskit-Qubit q_j.

Hinweis / Selbstkontrolle

Ein Test nur auf $|0^{n+1}\rangle$ ist unzureichend: Viele verschiedene Orakel besitzen dort denselben Ausgang.

Aufgabe 9 – Qiskit: Phase Kickback als Amplitudenvertrag

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 45 Minuten **Lernziel(e):** LZ1, LZ2, LZ4

Verwenden Sie das affine Orakel und präparieren Sie $|+\rangle^{\otimes n} |-\rangle$:

```python
def phase_kickback_circuit(secret: str, b: int = 0):
    n = len(secret)
    target = n
    qc = QuantumCircuit(n + 1)
    for j in range(n):
        qc.h(j)
    qc.x(target)
    qc.h(target)
    qc.compose(affine_oracle(secret, b), inplace=True)
    return qc
```

a) Leiten Sie den erwarteten Amplitudenvektor in Qiskit-Indexordnung her.

b) Erzeugen Sie ihn unabhängig:

```python
expected = np.zeros(2 * N, dtype=complex)
for x in range(N):
    phase = (-1) ** affine_value(x, secret, b)
    expected[x] = phase / np.sqrt(2 * N)
```

```
    expected[x + N] = -phase / np.sqrt(2 * N)
```

und vergleichen Sie mit `Statevector.from_instruction(qc).data`.

c) Prüfen Sie mindestens fünf zufällige Secrets und beide Offsets.

d) Wiederholen Sie mit Ziel $|+\rangle$. Welche Amplituden werden erwartet?

e) Wiederholen Sie mit Ziel $|0\rangle$ und bestimmen Sie für nichtkonstantes f die reduzierte Reinheit des Zielregisters.

f) Warum reicht `Statevector.equiv` bei einzelnen Basiszweigen nicht zur Prüfung relativer Phasen?

Hinweis / Selbstkontrolle

Mit Ziel $|+\rangle$ bleibt der Zustand unverändert. Ein globalphasenunempfindlicher Vergleich eines einzelnen Basiszweigs erkennt $(-1)^{f(x)}$ nicht.

Aufgabe 10 – Qiskit: Deutsch–Jozsa mit exakten Wahrscheinlichkeiten

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 45 Minuten **Lernziel(e):** LZ3–LZ5

Implementieren Sie:

```python
def dj_circuit(oracle: QuantumCircuit, n: int) -> QuantumCircuit:
    target = n
    qc = QuantumCircuit(n + 1)
    qc.x(target)
    for j in range(n + 1):
        qc.h(j)
    qc.compose(oracle, inplace=True)
    for j in range(n):
        qc.h(j)
    return qc
```

a) Testen Sie die konstanten Orakel $(s, b) = (0^n, 0)$ und $(0^n, 1)$.

b) Testen Sie für $n = 4$ mindestens fünf nichtleere lineare Secrets; diese Funktionen sind balanciert.

c) Berechnen Sie die Eingabewahrscheinlichkeiten mit

```
sv.probabilities_dict(qargs=list(range(n)))
```

und formulieren Sie einen automatischen Klassifikationstest.

d) Ziehen Sie mit festem Seed 128 Samples über `sample_counts`. Warum treten im idealen Modell keine abweichenden Strings auf?

e) Zeigen Sie für lineare balancierte Orakel, dass der konkrete Nichtnullausgang das Secret ist. Warum gilt dies nicht für allgemeine balancierte Funktionen?

f) Ergänzen Sie eine Promise-Prüfung für eine explizite Wahrheitstabelle.

Hinweis / Selbstkontrolle

Bei einer deterministischen Idealverteilung ist die Shot-Varianz null. Ein allgemeines balanciertes f kann seine Wahrscheinlichkeit auf mehrere Nichtnullstrings verteilen.

Aufgabe 11 – Qiskit: Bernstein–Vazirani für zufällige Secrets

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 45 Minuten **Lernziel(e):** LZ3–LZ5

Verwenden Sie denselben Schaltkreis, interpretieren Sie die Ausgabe nun als vollständigen String s.

a) Schreiben Sie `recover_secret(secret, b=0)`.

b) Testen Sie alle Secrets für $1 \leq n \leq 6$ und beide Werte b.

c) Prüfen Sie, dass die maximale Wahrscheinlichkeit numerisch 1 ist.

d) Vergleichen Sie die Zustandsvektoren für $b = 0$ und $b = 1$.

e) Testen Sie einen zufälligen String der Länge 20. Wie wachsen Queryzahl und CNOT-Zahl der offenen Orakelsynthese?

f) Formulieren Sie einen eigenschaftsbasierten Test statt eines fest codierten Beispiels.

Hinweis / Selbstkontrolle

Die beiden Offsets unterscheiden sich um globale Phase -1. Die Queryzahl bleibt eins; die Synthese benötigt $\mathrm{wt}(s)$ CNOTs.

Aufgabe 12 – Testgetriebene Orakelverträge und Mutationstests

Schwierigkeit: Aufbau **Arbeitsform:** Hybrid
Zeit: 55 Minuten **Lernziel(e):** LZ1, LZ4, LZ5

Entwerfen Sie eine Testsuite für `affine_oracle`. Sie soll prüfen:

a) vollständige Basiswahrheitstabelle für kleine n;

b) Unitarität und Involution;

c) Phase Kickback auf einer Superposition;

d) Deutsch–Jozsa für konstante und lineare balancierte Orakel;

e) Bernstein–Vazirani für zufällige Secrets;

f) dokumentierte Bitordnung.

Mutieren Sie einzeln:

i) `reversed(secret)` zu `secret`;

ii) ein `cx` zu einem falschen Kontrollqubit;

iii) die Zielpräparation;

iv) das Offset-X auf ein Eingabequbit;

v) das Orakel durch $(Z_{q_0} \otimes I)U_f$.

Dokumentieren Sie, welcher Vertrag welche Mutation erkennt. Ergänzen Sie eine Mutation, die ein reiner Basis-Messwahrscheinlichkeitstest nicht erkennt.

Hinweis / Selbstkontrolle

Die zusätzliche Z-Phase verändert keine klassische Wahrheitstabelle, aber Phase Kickback und Algorithmusausgabe.

Analyse, Debugging und Interpretation

Aufgabe 13 – Debugging: falscher Eigenzustand des Zielregisters

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 30 Minuten **Lernziel(e):** LZ2, LZ4, LZ5

Der folgende Bernstein–Vazirani-Schaltkreis liefert unabhängig vom Secret stets 0^n:

```python
def buggy_bv(secret: str):
    n = len(secret)
    target = n
    qc = QuantumCircuit(n + 1)
    for j in range(n + 1):
        qc.h(j)
    qc.compose(affine_oracle(secret), inplace=True)
    for j in range(n):
        qc.h(j)
    return qc
```

a) Bestimmen Sie den Zielzustand unmittelbar vor dem Orakel.

b) Berechnen Sie die Orakelwirkung algebraisch.

c) Leiten Sie den Eingabe-Endzustand her.

d) Korrigieren Sie den Code minimal.

e) Schreiben Sie einen Regressionstest, der den Bug bereits für ein Ein-Bit-Secret erkennt.

f) Warum reicht eine Prüfung von Breite und Gatteranzahl nicht?

Hinweis / Selbstkontrolle

Ohne vorheriges X wird das Ziel zu $|+\rangle$. Dieser Zustand besitzt unter X Eigenwert $+1$ und erzeugt kein Phasensignal.

Aufgabe 14 – Debugging: Wahrheitstabelle korrekt, Quantenphase falsch

Schwierigkeit: Aufbau **Arbeitsform:** Hybrid
Zeit: 40 Minuten **Lernziel(e):** LZ1, LZ2, LZ5

Ein Entwickler ersetzt ein korrektes Orakel durch

$$U_f' = (Z_{q_0} \otimes I_{\text{rest}})U_f.$$

Er testet nur Rechnerbasis-Eingaben und vergleicht gemessene Basislabels. Alle Tests bestehen.

a) Zeigen Sie, warum die Basis-Messverteilungen von U_f und U_f' gleich sind.

b) Zeigen Sie, dass die Operatoren nicht gleich sind.

c) Berechnen Sie ihre Wirkung auf $|+\rangle_{q_0}|0\cdots0\rangle|-\rangle$.

d) Welche Änderung erfährt das Bernstein–Vazirani-Secret?

e) Ergänzen Sie einen Interferenztest, der den Fehler sicher erkennt.

f) Welche Informationen prüft ein reiner Wahrheitstabellentest nicht?

Hinweis / Selbstkontrolle

Das zusätzliche Z multipliziert alle Zweige mit $x_0 = 1$ um -1. Im Walsh-Spektrum entspricht dies einer XOR-Verschiebung um e_0.

Aufgabe 15 – Debugging: Secret-String und Little Endian

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 35 Minuten **Lernziel(e):** LZ4, LZ5
Ein Orakelbuilder verwendet

```
for j, bit in enumerate(secret):
    if bit == "1":
        oracle.cx(j, target)
```

Der gewünschte String sei `"10110"`, geschrieben als $s_4 s_3 s_2 s_1 s_0$.

a) Welcher mathematische String wird tatsächlich implementiert?

b) Welches BV-Ergebnis ist zu erwarten?

c) Korrigieren Sie die Schleife auf zwei Arten.

d) Schreiben Sie einen Test für alle Einheitssecrets.

e) Warum kann ein palindromisches Testsecret den Fehler verbergen?

f) Formulieren Sie eine explizite Namenskonvention für String- und Qubitreihenfolge.

Hinweis / Selbstkontrolle

Das linke Zeichen wird fälschlich q_0 zugeordnet. Der implementierte angezeigte String ist die Umkehrung des gewünschten Strings.

Aufgabe 16 – Promise-Verletzung, Shots und Auslesefehler

Schwierigkeit: Transfer **Arbeitsform:** Hybrid
Zeit: 45 Minuten **Lernziel(e):** LZ3–LZ6
Ein Deutsch–Jozsa-Experiment verwendet $n = 4$.

a) Eine Funktion besitzt genau $m = 3$ Einsen. Berechnen Sie die ideale Wahrscheinlichkeit $p(0000)$.

b) Simulieren Sie die vollständige Walsh-Verteilung für eine konkrete Funktion mit drei Einsen und ziehen Sie 100, 1000 und 10000 Samples.

c) Ein konstantes ideales Orakel liefert vor der Auslese deterministisch 0000. Jedes Bit werde unabhängig mit Wahrscheinlichkeit $e = 0.02$ vertauscht. Berechnen Sie $P(0000)$.

d) Berechnen Sie die Wahrscheinlichkeit für genau einen Bitfehler.

e) Simulieren Sie die Ausleseverteilung für 10000 Shots.

f) Warum entscheidet eine beobachtete Häufigkeit von 0.91 für 0000 nicht eindeutig über die Ursache?

g) Entwerfen Sie ein Diagnoseprotokoll, das Statevector, ideale Shots und Fehlermodell trennt.

Hinweis / Selbstkontrolle

$p(0000) = 25/64$. Im unabhängigen Auslesemodell gilt $P(0000) = (1-e)^4$ und $P(\text{genau ein Fehler}) = 4e(1-e)^3$.

Transfer, Abfragemodelle und physikalische Anwendung

Aufgabe 17 – Randomisierte klassische Baseline für Deutsch–Jozsa

Schwierigkeit: Transfer **Arbeitsform:** Hybrid
Zeit: 45 Minuten **Lernziel(e):** LZ3, LZ6

Ein klassischer Algorithmus wählt k Eingaben unabhängig und gleichverteilt. Sobald zwei verschiedene Funktionswerte erscheinen, meldet er „balanciert“, andernfalls „konstant“.

a) Zeigen Sie, dass er bei einer konstanten Funktion nie irrt.

b) Leiten Sie für balanciertes f die Fehlerwahrscheinlichkeit 2^{1-k} her.

c) Bestimmen Sie das kleinste k für 10^{-2}, 10^{-6} und 2^{-40}.

d) Simulieren Sie die Fehlerhäufigkeit für verschiedene n und k.

e) Vergleichen Sie die Abhängigkeit von n mit $2^{n-1}+1$.

f) Formulieren Sie eine korrekte Vorteilsaussage mit Rechenmodell und Fehlertoleranz.

Hinweis / Selbstkontrolle

Für feste Fehlertoleranz ist die randomisierte Queryzahl unabhängig von n. Dies verwendet ein anderes Vergleichsmodell als die exakte deterministische Trennung.

Aufgabe 18 – Eine Oracle Query und ihre Schaltkreiskosten

Schwierigkeit: Transfer **Arbeitsform:** Papier
Zeit: 45 Minuten **Lernziel(e):** LZ1, LZ4, LZ6

Ein BV-Orakel besitzt sechs Eingabequbits $q_0, \ldots, q_5$ und Ziel q_6. Aktiv sind $j \in \{0, 3, 5\}$.

a) Wie viele CNOTs benötigt die abstrakte all-to-all Zerlegung?

b) Die Kopplung sei die Linie $q_0 - q_1 - \cdots - q_6$. Für jedes aktive j wird das Kontrollqubit bis neben q_6 geswappt, die CNOT ausgeführt und die Platzierung zurückgesetzt. Leiten Sie bei Abstand $d = 6 - j$ die Kosten $6(d-1)+1$ her.

c) Berechnen Sie die Gesamtkosten.

d) Vergleichen Sie „eine Query“ mit der nativen Zwei-Qubit-Gatterzahl.

e) Nehmen Sie illustrativ Erfolgswahrscheinlichkeit 0.995 je Zwei-Qubit-Gatter an. Vergleichen Sie die Produktmodelle für drei und für die berechnete Zahl CNOTs.

f) Nennen Sie mindestens fünf Grenzen dieses Produktmodells.

g) Skizzieren Sie eine bessere gemeinsame Routingstrategie.

Hinweis / Selbstkontrolle

Die naive rückgesetzte Strategie benötigt $31 + 13 + 1 = 45$ CNOTs. Dies ist ein Synthesemodell, kein Hardwarebenchmark.

Aufgabe 19 – Physikalisches Phase Kickback durch Eigenphasen

Schwierigkeit: Transfer **Arbeitsform:** Hybrid
Zeit: 50 Minuten **Lernziel(e):** LZ2, LZ4, LZ6

Ein Kontrollqubit C steuert auf Ziel T die Operation

$$P(\varphi) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\varphi} \end{pmatrix}.$$

Das Ziel startet in $|1\rangle_T$, die Kontrolle in $|+\rangle_C$.

a) Zeigen Sie, dass $|1\rangle_T$ Eigenzustand mit Eigenwert $e^{i\varphi}$ ist.

b) Leiten Sie den Ausgang der kontrollierten Operation her und zeigen Sie, dass das Ziel separiert bleibt.

c) Wenden Sie H auf die Kontrolle an und leiten Sie

$$p_C(0) = \cos^2\frac{\varphi}{2}, \qquad p_C(1) = \sin^2\frac{\varphi}{2}$$

her.

d) Wiederholen Sie für Ziel $|0\rangle_T$.

e) Wiederholen Sie für Ziel $|+\rangle_T$ und zeigen Sie, dass im Allgemeinen Verschränkung entsteht.

f) Implementieren Sie einen Qiskit-Scan für $0 \le \varphi \le 2\pi$.

g) Interpretieren Sie $P(\varphi)$ als bedingte Phasenakkumulation eines Zweiniveausystems. Welche Annahme ist für sauberes Kickback entscheidend?

Hinweis / Selbstkontrolle

Entscheidend ist, dass das Ziel in einem Eigenzustand der bedingt angewandten Operation präpariert ist.

Aufgabe 20 – Primärquellen: Mustervergleich früher Quantenalgorithmen

Schwierigkeit: Transfer **Arbeitsform:** Literatur
Zeit: 90 Minuten **Lernziel(e):** LZ3, LZ6

Lesen Sie ausgewählte Abschnitte aus Deutsch–Jozsa (1992), Bernstein–Vazirani (1997) und Cleve et al. (1998). Erstellen Sie einen zweiseitigen mathematischen Mustervergleich mit:

a) Problemdefinition und Promise;

b) klassischem und quantummechanischem Abfragemodell;

c) Orakelabbildung;

d) Präparation von Eingabe und Ziel;

e) Phase Kickback;

f) ausgelesenem Walsh–Hadamard-Koeffizienten;

g) idealer Messentscheidung;

h) exakter klassischer und quantummechanischer Query-Komplexität;

i) Rolle von Randomisierung und Fehlertoleranz;

j) mindestens drei Grenzen der Übertragung auf reale Anwendungen.

Markieren Sie jede Aussage als Primärquellenbefund, eigene Herleitung oder heutige Einordnung.

Hinweis / Selbstkontrolle

Ein historischer Geschwindigkeitssatz darf nicht ohne Vergleichsmodell wiederholt werden. Trennen Sie exakte deterministische, randomisierte bounded-error und end-to-end Kosten.

Literatur und Vertiefung

David Deutsch und Richard Jozsa: “Rapid solution of problems by quantum computation”, *Proceedings of the Royal Society A* 439, 553–558 (1992), DOI: 10.1098/rspa.1992.0167.

Für ursprüngliche Problemformulierung und historische Vorteilsaussage; Aufgaben 5, 10, 16, 17 und 20.

Ethan Bernstein und Umesh Vazirani: "Quantum Complexity Theory", *SIAM Journal on Computing* 26(5), 1411–1473 (1997), DOI: 10.1137/S0097539796300921.
Für verborgenes lineares Bitstringproblem und Query-Komplexität; Aufgaben 6, 11, 15, 18 und 20.

Richard Cleve, Artur Ekert, Chiara Macchiavello und Michele Mosca: "Quantum algorithms revisited", *Proceedings of the Royal Society A* 454, 339–354 (1998), DOI: 10.1098/rspa.1998.0164.
Für moderne Schaltkreis- und Interferenzdarstellung; Aufgaben 3, 5, 6 und 20.

Michael A. Nielsen und Isaac L. Chuang: *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press, 2010.
Mathematische Referenz für reversible Orakel, kontrollierte Operationen und frühe Quantenalgorithmen.

IBM Quantum Documentation: *Statevector*.
`https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector`
API-Referenz für `from_instruction`, `from_int`, `evolve`, Wahrscheinlichkeiten und `sample_counts`.

IBM Quantum Documentation: *Operator*.
`https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Operator`
Für exakte Operatorprüfungen, Unitarität und Involution.

IBM Quantum Documentation: *Bit ordering in Qiskit*.
`https://quantum.cloud.ibm.com/docs/en/guides/bit-ordering`
Für String-, Integer-, Qubit- und Anzeigekonvention in Aufgaben 8–15.